Semantic Segmentation in Pontryagin Spaces

Random Feature Embeddings with Indefinite Metric for UAV Crop Imagery

Carlos Jiménez

April 17, 2026

Contents

1 Introduction

Deep neural network classifiers typically embed intermediate representations in Euclidean space and apply a linear decision rule. A growing body of work has shown that certain data distributions — in particular those with latent hierarchical or hyperbolic structure — are better captured by non-Euclidean geometries. Hyperbolic spaces (Poincaré ball, hyperboloid) have been explored for this purpose [**?**, **?**], but they impose a *constant negative* curvature that may not suit all problems.

This work proposes an alternative: embedding pixel features into a **Pontryagin space** Π_κ, a flat pseudo-Riemannian manifold equipped with an *indefinite* inner product of signature $(\cdot, \kappa)$. Unlike the Poincaré ball, a Pontryagin space is both flat and able to represent mixed-curvature structure through the coexistence of positive- and negative-definite subspaces. The embedding is constructed from two families of random features:

- **Random Fourier Features (RFF)** [**?**] approximate a positive-definite RBF kernel K_+.
- **Spherical Random Features (SRF)** [**?**] approximate a polynomial kernel K_- of degree d_{poly} on the unit sphere.

The resulting feature vector lives in $\Pi_\kappa = \mathbb{R}^{p+\kappa}$ with metric $J = \text{diag}(+1^p, -1^\kappa)$, and classification is performed by *J-hyperplane* multinomial logistic regression.

The experimental testbed is binary semantic segmentation of sugarcane crops from UAV imagery, where all three heads (Euclidean, Hyperbolic, Pontryagin) share an identical deep FCN backbone and training protocol, isolating the effect of the embedding geometry. Interpretability of each model is assessed using **SegScoreCAM**, an adaptation of ScoreCAM for pixel-wise segmentation outputs.

2 Dataset

2.1 UAV Sugarcane Segmentation

The dataset consists of aerial images acquired by an unmanned aerial vehicle (UAV) over agricultural fields containing sugarcane, banana trees, pepper, spinach, and other crops. The annotation format is **YOLO polygon segmentation**: each label file contains class index and normalised polygon vertices per instance.

Preprocessing. All images are resized to 512×512 pixels using bilinear interpolation. Polygon masks are rasterised at the same resolution. To focus on a single, well-defined binary problem, the dataset is filtered to **binary sugarcane segmentation**: class 0 = background, class 1 = sugarcane; images without any sugarcane polygon are discarded. Pixel intensities are normalised with ImageNet statistics ($\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$).

Class imbalance. Background pixels are far more frequent than sugarcane pixels. A weighted cross-entropy loss compensates for this imbalance with weights $w_{\text{bg}} = 0.20$ and $w_{\text{sc}} = 1.00$.

Splits and augmentation. The dataset is split into train / validation / test partitions provided by the source repository. During training, random horizontal and vertical flips (each with probability 0.5) are applied.

3 Backbone Architecture

3.1 DeepFCN

All three segmentation heads share a **deep fully-convolutional network (DeepFCN)** backbone with configurable depth $D \in \{3, 4, 5\}$. Each stage consists of two 3×3 strided convolutions (stride 2) with batch normalisation and ReLU activation. Channel widths follow the schedule $3 \to 32 \to 64 \to 128 \to 256 \to 256$ (the last stage is repeated when $D = 5$). A final 1×1 projection maps the last stage to a fixed $C_{\text{out}} = 64$ channel dimension.

Given an input $\mathbf{x} \in \mathbb{R}^{3\times H\times W}$, the backbone produces a feature map $\mathbf{f} \in \mathbb{R}^{64\times H'\times W'}$ where $H' = H/2^D$. This map is then processed by the segmentation head, and the resulting low-resolution logits are bilinearly upsampled to the original resolution before computing the loss.

4 Segmentation Heads

4.1 Euclidean (Baseline)

The Euclidean head applies a 1×1 convolution directly to the backbone features:

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{cls}}\,\mathbf{f} + \mathbf{b}, \quad \mathbf{W}_{\text{cls}} \in \mathbb{R}^{C\times 64}.$$

This is the standard FCN classifier; no geometric embedding is applied.

4.2 Hyperbolic (Ganea / Atigh Baseline)

The hyperbolic head maps backbone features into the Poincaré ball $\mathbb{B}_c^d$ of curvature c using a Möbius transformation, then applies the hyperbolic multinomial logistic regression (MLR) of Ganea et al. [**?**] extended to the pixel level by Atigh et al. [**?**]. The decision boundary for class y is a *geodesic hyperplane* in $\mathbb{B}_c^d$.

The curvature c is a hyperparameter optimised during search.

4.3 Pontryagin (This Work)

Motivation. The Poincaré ball has constant negative curvature, which biases the space towards tree-like structures. A Pontryagin space Π_κ is flat but has an *indefinite* metric; it can simultaneously encode positive- and negative-curvature regions through the balance of its positive and negative subspace dimensions.

Definition 1 (Pontryagin space Π_κ)**.** A Pontryagin space of signature κ is a real vector space $\mathbb{R}^{p+\kappa}$ equipped with the indefinite inner product

$$\langle \mathbf{u}, \mathbf{v}\rangle_J = \sum_{i=1}^{p} u_i v_i - \sum_{i=p+1}^{p+\kappa} u_i v_i = \mathbf{u}^\top J\,\mathbf{v}, \quad J = \operatorname{diag}(\underbrace{+1, \ldots, +1}_{p}, \underbrace{-1, \ldots, -1}_{\kappa}).$$

Random feature construction. Given a backbone feature vector $\mathbf{f} \in \mathbb{R}^{64}$ at each pixel, the Pontryagin embedding is built as follows.

Positive component — RFF. Random Fourier Features [?] approximate the RBF kernel

$$K_+(\mathbf{x}, \mathbf{y}) = \exp\left(-\tfrac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2}\right).$$

Frequencies $\omega_i \sim \mathcal{N}(0, \sigma^{-2} I)$ and phases $b_i \sim \text{Uniform}[0, 2\pi]$ are sampled once; the feature map is

$$\varphi_+(\mathbf{x}) = \frac{1}{\sqrt{n_{\text{rff}}}} \Big[\cos(\omega_1^\top \mathbf{x} + b_1),\ \sin(\omega_1^\top \mathbf{x} + b_1),\ \ldots\Big] \in \mathbb{R}^{2n_{\text{rff}}}.$$

The bandwidth σ is stored as a learnable parameter $\log \sigma$, which is updated by gradient descent and therefore *removed from the HPO space.*

Negative component — SRF. Spherical Random Features [?] approximate the polynomial kernel

$$K_-(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^\top \mathbf{y})^{d_{\text{poly}}},$$

where d_{poly} is the polynomial degree (*independent of the signature* κ). The j-th feature is

$$\varphi_-(\mathbf{x})_j = \frac{1}{\sqrt{\kappa}} \prod_{\ell=1}^{d_{\text{poly}}} \big(\omega_{j\ell}^\top \mathbf{x}\big), \quad \omega_{j\ell} \sim \text{Uniform}(S^{d-1}),$$

producing $\varphi_-(\mathbf{x}) \in \mathbb{R}^\kappa$.

Concatenation. The Pontryagin embedding is

$$\Phi(\mathbf{x}) = \big[\varphi_+(\mathbf{x}) \,\|\, \varphi_-(\mathbf{x})\big] \in \mathbb{R}^{p+\kappa}, \quad p = 2\, n_{\text{rff}}.$$

The metric matrix $J = \text{diag}(+1^p, -1^\kappa)$ is stored as a fixed buffer.

Remark 1. The signature κ and the polynomial degree d_{poly} are **decoupled hyperparameters**: κ sets how many negative directions exist in J (and therefore how many SRF components are produced), while d_{poly} controls the nonlinearity of the kernel K_-. Both are optimised independently in the HPO search.

J-hyperplane classification. For each class y, the logit is the indefinite inner product with a learnable weight vector $\mathbf{W}_y \in \mathbb{R}^{p+\kappa}$:

$$\zeta_y(\mathbf{z}) = \langle \mathbf{W}_y,\, \mathbf{z} \rangle_J + b_y = \sum_{d=1}^{p+\kappa} J_d\, W_{y,d}\, z_d + b_y.$$

5 Loss Function

The Pontryagin head minimises a composite loss:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_W\, \mathcal{L}_{\text{cone-W}} + \lambda_{\text{topo}}\, \mathcal{L}_{\text{topo}} + \lambda_{\text{bal}}\, \mathcal{L}_{\text{balance}}.$$

Cross-entropy.

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{n=1}^{N} \log \frac{e^{\zeta_{y_n}(\mathbf{z}_n)}}{\sum_c e^{\zeta_c(\mathbf{z}_n)}},$$

weighted by class frequencies.

Cone penalty on weights $\mathcal{L}_{\text{cone-W}}$. Weight vectors that lie on the *null cone* $\{u : \langle u, u\rangle_J = 0\}$ produce degenerate classifiers (the J-hyperplane has no definite normal). The penalty keeps them away from the cone:

$$\mathcal{L}_{\text{cone-W}} = \frac{1}{C}\sum_{y=1}^{C} \text{ReLU}\big(\varepsilon - \big|\mathbf{W}_y^\top J\, \mathbf{W}_y\big|\big)\,, \quad \varepsilon = 0.1.$$

Topographic penalty $\mathcal{L}_{\text{topo}}$. The `IndefiniteTopographicPenalty` combines three terms on the embedding $\mathbf{z}$:

- *Light-cone (LC):* penalises embeddings near the null cone.
- *Causal consistency (CC):* encourages same-class embeddings to share the same causal character (timelike or spacelike).
- *Signature balance (SB):* keeps the norms of the positive and negative subcomponents similar.

Balance penalty $\mathcal{L}_{\text{balance}}$.

$$\mathcal{L}_{\text{balance}} = \text{ReLU}(\|\mathbf{z}_-\|_2 - \|\mathbf{z}_+\|_2)\,,$$

where $\mathbf{z}_+$ and $\mathbf{z}_-$ are the positive and negative subspace components. This prevents the negative subspace from dominating the representation.

6 Training Protocol

Optimiser. Adam with learning rate $\eta = 10^{-3}$ and weight decay 10^{-4}.

Scheduler. Cosine annealing from η to $\eta_{\min} = 10^{-5}$ over $E_{\max} = 60$ epochs.

Early stopping. Training halts if the validation *macro mIoU* (averaged over non-empty classes) does not improve for 15 consecutive epochs. The checkpoint with the highest validation macro mIoU is kept.

Batch size. 2 images per GPU step (limited by the 512^2 resolution and GPU memory).

Hardware. Single CUDA GPU.

7 Hyperparameter Optimisation

Bayesian HPO is carried out with `scikit-optimize`'s `forest_minimize` (Extra Trees surrogate, 30 trials, 10 random starts). Each trial trains for 12 epochs with early stopping at patience 6. The search spaces are model-specific.

7.1 Common to all models

Parameter	Range	Prior
`lr`	$[10^{-5},\ 10^{-2}]$	log-uniform
`weight_decay`	$[10^{-6},\ 10^{-3}]$	log-uniform
`backbone_depth`	$\{3, 4, 5\}$	uniform

7.2 Hyperbolic-specific

Parameter	Range	Prior
`hyperbolic_c`	$[0.1,\ 2.0]$	log-uniform

7.3 Pontryagin-specific

Parameter	Range	Interpretation
κ (`kappa`)	$\{1, 2, 3, 4\}$	Pontryagin signature $= n_{\text{srf}}$
d_{poly} (`d_poly`)	$\{1, 2, 3, 4\}$	SRF polynomial degree
`rff_multiplier`	$\{1, 2, 3, 4\}$	$n_{\text{rff}} = m \cdot C_{\text{out}}$
λ_W (`lambda_cone_W`)	$[10^{-6},\ 0.5]$	cone penalty weight
λ_{topo}	$[10^{-6},\ 0.2]$	topographic penalty weight
λ_{bal}	$[10^{-6},\ 0.1]$	balance penalty weight

Note: the RFF bandwidth σ is *not* in the HPO space because it is optimised by gradient descent via the reparametrisation $\sigma = \exp(\log \sigma)$.

8 Evaluation Metrics

All metrics are computed on the held-out test split.

Per-class metrics. For each class c: true positives (TP_c), false positives (FP_c), false negatives (FN_c), and true negatives (TN_c); then

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}, \qquad \text{Dice}_c = \frac{2\,\text{TP}_c}{2\,\text{TP}_c + \text{FP}_c + \text{FN}_c}.$$

Precision, recall, and specificity (TNR) are also reported.

Global metrics.

- *Macro mIoU / Dice*: average over non-empty classes.
- *Micro IoU / Dice*: pooled across all pixels.
- *Pixel accuracy*: fraction of correctly labelled pixels.

The monitor criterion for early stopping and HPO is the **validation macro mIoU**.

9 Interpretability: SegScoreCAM

9.1 Motivation

The original ScoreCAM [**?**] was designed for image *classification*: the score for each feature-map channel is the scalar class logit after masking. Segmentation heads produce spatial output maps $\hat{P} \in \mathbb{R}^{C \times H \times W}$; extracting a single logit discards all spatial information.

9.2 SegScoreCAM

We replace the scalar score with the **mean softmax probability** of the target class over all spatial positions:

$$s_k = \frac{1}{HW} \sum_{i,j} \mathrm{softmax}\big(f_\theta(I \odot \tilde{A}_k)\big)_{c,i,j},$$

where $\tilde{A}_k$ is the k-th feature map normalised to $[0, 1]$ and upsampled to input resolution, and $I \odot \tilde{A}_k$ is the masked input.

The class activation map for class c is then:

$$\mathrm{CAM}_c = \mathrm{ReLU}\left(\sum_k \bar{s}_k \cdot A_k\right), \quad \bar{s}_k = \frac{|s_k|}{\sum_{k'} |s_{k'}|},$$

followed by min-max normalisation to $[0, 1]$.

9.3 Interpretability Metrics

For the sugarcane class (class 1), three metrics quantify the quality of the heatmap against the ground-truth mask M:

Metric	Definition
GT IoU	$\mathrm{IoU}(\mathbf{1}[\mathrm{CAM} \geq 0.5],\ M)$
GT Recall	fraction of GT pixels covered by $\mathbf{1}[\mathrm{CAM} \geq 0.5]$
Mean activation	$\frac{1}{HW} \sum_{i,j} \mathrm{CAM}_{i,j}$ (lower = more focused)

9.4 Interpretability Results

Table 1: SegScoreCAM metrics on the test split (mean ± std). GT IoU and GT Recall are higher-is-better; Mean Activation is lower-is-better (a smaller value indicates a more focused, less diffuse heatmap).

Model	GT IoU ↑	GT Recall ↑	Mean Activation ↓
Euclidean	0.287 ± 0.282	0.300 ± 0.294	0.054 ± 0.065
Hyperbolic	0.280 ± 0.248	0.300 ± 0.269	0.056 ± 0.064
Pontryagin	0.110 ± 0.273	0.154 ± 0.327	0.366 ± 0.101

The Euclidean head achieves the best GT IoU and GT Recall, closely followed by the hyperbolic head. The Pontryagin model shows a significantly higher mean activation

(0.366 vs ∼0.055), indicating that its heatmaps are more diffuse and less localised on the sugarcane region. This behaviour warrants further investigation; it may reflect that the indefinite metric disperses the activation across a wider receptive field.

10 Discussion

Pontryagin vs Euclidean. The decoupling of the polynomial degree d_{poly} from the Pontryagin signature κ enables a cleaner ablation of two distinct design choices: (i) the dimensionality of the negative subspace and (ii) the degree of the polynomial kernel K_-. Earlier implementations tied these two values, making it impossible to determine which factor drove performance.

Signature and degree. Small values of κ (e.g., $\kappa = 1$ or 2) introduce a modest indefinite component that may capture local curvature without dominating the representation. The polynomial degree d_{poly} controls how strongly the negative component deviates from linearity: $d_{\text{poly}} = 1$ recovers a linear kernel on the sphere, while $d_{\text{poly}} \geq 2$ introduces genuine nonlinearity in K_-.

Interpretability. The diffuse SegScoreCAM heatmaps produced by the Pontryagin model suggest that its features are less spatially concentrated than those of the Euclidean or hyperbolic heads. One hypothesis is that the balance penalty $\mathcal{L}_{\text{balance}}$, combined with the topographic penalty, pushes embeddings away from localised configurations.

11 Conclusion

We presented a semantic segmentation framework based on Pontryagin space embeddings, combining Random Fourier Features (RFF) and Spherical Random Features (SRF) with an indefinite J-metric and a J-hyperplane classifier. Three heads — Euclidean, Hyperbolic, and Pontryagin — were trained on a UAV sugarcane segmentation dataset under a controlled protocol (identical backbone, optimiser, and schedule).

Key contributions are:

1. A pixel-level Pontryagin MLR head with composite loss (CE + cone + topographic + balance penalties).
2. Decoupling of the Pontryagin signature κ from the SRF polynomial degree d_{poly}, enabling independent HPO of both.
3. SegScoreCAM: an adaptation of ScoreCAM for segmentation models using mean spatial softmax probability as the channel score.
4. An interpretability comparison showing that the Euclidean and hyperbolic heads produce more focused heatmaps than the Pontryagin head at the current hyperparameter settings.

Future work includes a full ablation over $(\kappa, d_{\text{poly}})$ combinations, analysis of the embedding geometry via t-SNE/UMAP in Pontryagin space, and extension to multi-class segmentation.

References

[1] A. Rahimi and B. Recht. *Random Features for Large-Scale Kernel Machines*. NeurIPS, 2007.

[2] J. Pennington, F. Yu, and S. Kumar. *Spherical Random Features for Polynomial Kernels*. NeurIPS, 2015.

[3] O.-E. Ganea, G. Bécigneul, and T. Hofmann. *Hyperbolic Neural Networks*. NeurIPS, 2018.

[4] M. G. Atigh, J. Schoep, E. Acar, N. van Noord, and P. Mettes. *Hyperbolic Image Segmentation*. CVPR, 2022.

[5] H. Wang, Z. Wang, M. Du, F. Yang, Z. Zhang, S. Ding, P. Mardziel, and X. Hu. *Score-CAM: Score-Weighted Visual Explanations for Convolutional Neural Networks*. CVPR Workshops, 2020.